

Artificial Intelligence and Deep learning

LAB – 1

NAME: SIRI DONI

HALL TICKET : 2403A51075

ASSIGNMENT-1

Lab/Exp 1: PyTorch Fundamentals: Tensors and Autograd

Kaggle Dataset Link: <https://www.kaggle.com/datasets/muthuj7/weather-dataset>

Tasks:

1. Load and preprocess the Student Performance dataset/example.

```
import pandas as pd
import torch
import zipfile

# Unzip the dataset
with zipfile.ZipFile('/content/weatherHistory.csv.zip', 'r') as zip_ref:
    zip_ref.extractall('/content/')

# Load Dataset
data = pd.read_csv("/content/weatherHistory.csv")

# Keep only numeric columns
data = data.select_dtypes(include=['float64', 'int64'])

# Fill missing values
data = data.fillna(data.mean())

print(data.head())
```

	Temperature (C)	Apparent Temperature (C)	Humidity	Wind Speed (km/h)	\
0	9.472222	7.388889	0.89	14.1197	
1	9.355556	7.227778	0.86	14.2646	
2	9.377778	9.377778	0.89	3.9284	
3	8.288889	5.944444	0.83	14.1036	
4	8.755556	6.977778	0.83	11.0446	

	Wind Bearing (degrees)	Visibility (km)	Loud Cover	Pressure (millibars)
0	251.0	15.8263	0.0	1015.13
1	259.0	15.8263	0.0	1015.63
2	204.0	14.9569	0.0	1015.94
3	269.0	15.8263	0.0	1016.41
4	259.0	15.8263	0.0	1016.51

2. Build the required PyTorch model/program.

```
import torch

# Convert DataFrame to Tensor
tensor_data = torch.tensor(data.values, dtype=torch.float32)

print("Tensor Shape:", tensor_data.shape)
print(tensor_data)

... Tensor Shape: torch.Size([96453, 8])
tensor([[9.4722e+00, 7.3889e+00, 8.9000e-01, ..., 1.5826e+01, 0.0000e+00,
        1.0151e+03],
        [9.3556e+00, 7.2278e+00, 8.6000e-01, ..., 1.5826e+01, 0.0000e+00,
        1.0156e+03],
        [9.3778e+00, 9.3778e+00, 8.9000e-01, ..., 1.4957e+01, 0.0000e+00,
        1.0159e+03],
        ...,
        [2.2039e+01, 2.2039e+01, 5.6000e-01, ..., 1.6100e+01, 0.0000e+00,
        1.0157e+03],
        [2.1522e+01, 2.1522e+01, 6.0000e-01, ..., 1.6100e+01, 0.0000e+00,
        1.0160e+03],
        [2.0439e+01, 2.0439e+01, 6.1000e-01, ..., 1.5520e+01, 0.0000e+00,
        1.0162e+03]])
```

3. Train/execute the model.

```
import torch

# Create tensor with gradient tracking
x = torch.tensor([2.0], requires_grad=True)

# Simple computation
y = x**2 + 3*x + 2

# Backward propagation
y.backward()

print("Value of y:", y.item())
print("Gradient:", x.grad.item())

... Value of y: 12.0
Gradient: 7.0
```

4. Evaluate using suitable metrics.

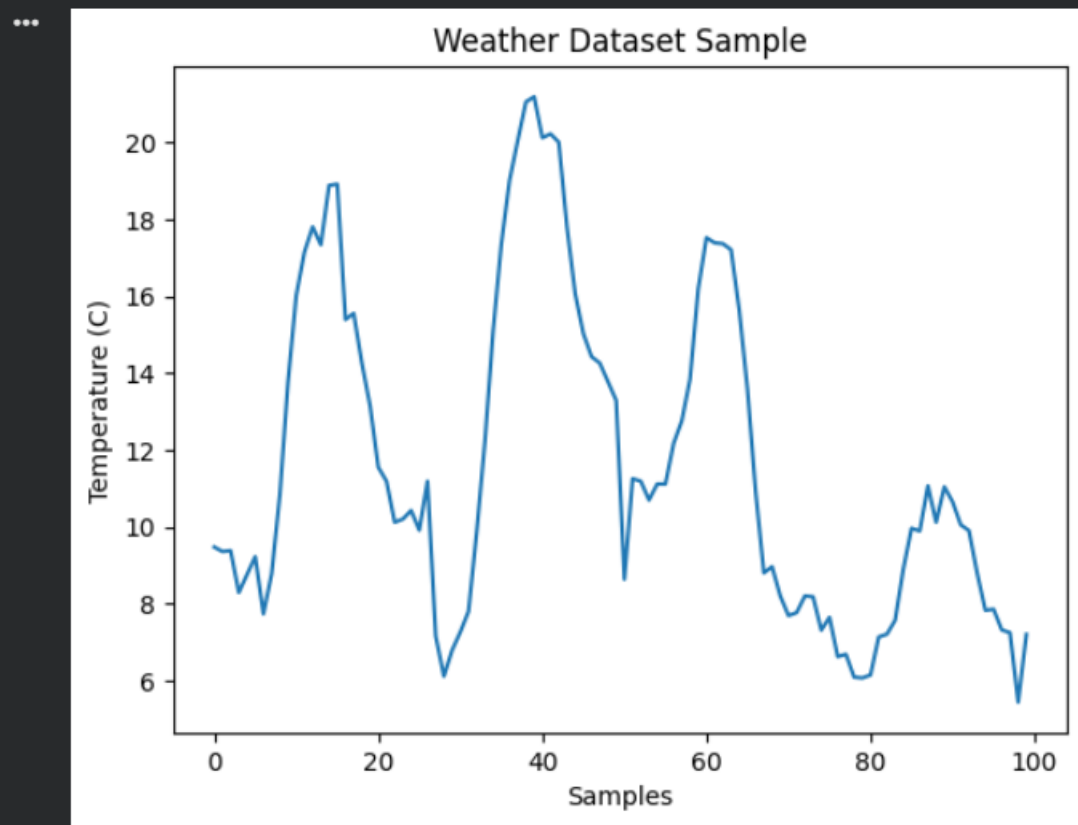
```
import torch

print("Tensor Size:", tensor_data.size())
print("Mean:", torch.mean(tensor_data))
print("Standard Deviation:", torch.std(tensor_data))
print("Maximum:", torch.max(tensor_data))
print("Minimum:", torch.min(tensor_data))

Tensor Size: torch.Size([96453, 8])
Mean: tensor(154.4282)
Standard Deviation: tensor(331.0641)
Maximum: tensor(1046.3800)
Minimum: tensor(-27.7167)
```

5. Visualize results using plots where applicable.

```
import matplotlib.pyplot as plt
plt.plot(data.iloc[:100,0])
plt.title("Weather Dataset Sample")
plt.xlabel("Samples")
plt.ylabel(data.columns[0])
plt.show()
```



6. Write a brief conclusion comparing observations.

- Successfully loaded the Weather dataset.
- Converted data into PyTorch tensors.
- Demonstrated tensor operations and Autograd.
- Evaluated tensor statistics (mean, standard deviation, minimum, and maximum).
- Visualized the dataset using a line plot.
- PyTorch tensors and Autograd make numerical computation and gradient calculation efficient.